

Github: <https://github.com/ys6-149286/BigData2026-Assignment1>

Question 1:

Report the accuracy of original Llama-3.2-1B on both GSM8K and MATH datasets.

Fine-tune Llama-3.2-1B by setting the target rank to 16, 64, and 256, respectively, and report the accuracy on both GSM8K and MATH datasets.

Provide your observations and explanations on the performance trends with different target ranks for both datasets.

Accuracy	Original LLM	Fine-Tuned LLM (RANK=16)	Fine-Tuned LLM (RANK=64)	Fine-Tuned LLM (RANK=256)
GSM8K	0.035	0.157	0.206	0.270
MATH	0.001	0.038	0.047	0.055

→ See https://github.com/ys6-149286/BigData2026-Assignment1/blob/kaggle_nb/q1q2_notebook.ipynb

→ The larger ranks tend to get higher accuracy after fine-tuning. Indicating that larger rank might have a better initial status in the search spaces of trainable parameters.

→ We can see 3 fine-tuned runs got significant better accuracy compared to the baseline. That is, fine-tuning with relative datasets (MetaMath) can easily improve the model capability with similar downstream tasks (GSM8K, MATH).

Question 2:

a. Following Question 1, run experiments with target ranks of 16, 64, and 256, and record the **training time (minute)** for each setting.

→ See https://github.com/ys6-149286/BigData2026-Assignment1/blob/kaggle_nb/q1q2_notebook.ipynb

```
metamath_pissa_default_r16
=====
Loading weights: 100% 146/146 [00:01<00:00, 146.08it/s]
[ 1/112] model.layers.0.self_attn.q_proj
[ 2/112] model.layers.0.self_attn.k_proj
[ 3/112] model.layers.0.self_attn.v_proj
[112/112] model.layers.15.mlp.down_proj
Trainable parameters: 11,272,192 (0.904%)
Training metamath: 24975 examples, 3122 micro-batches, 196 updates, 6 warmup updates.
Train metamath epoch 1/1: 100% 3122/3122 [1:04:58<00:00, 1.26s/it, loss=0.4547, update=196]
```

```
metamath_pissa_default_r64
=====
Loading weights: 100% 146/146 [00:01<00:00, 149.56it/s]
[ 1/112] model.layers.0.self_attn.q_proj
[ 2/112] model.layers.0.self_attn.k_proj
[ 3/112] model.layers.0.self_attn.v_proj
[112/112] model.layers.15.mlp.down_proj
Trainable parameters: 45,088,768 (3.520%)
Training metamath: 24975 examples, 3122 micro-batches, 196 updates, 6 warmup updates.
Train metamath epoch 1/1: 100% 3122/3122 [1:09:33<00:00, 1.35s/it, loss=0.4140, update=196]
```

```
metamath_pissa_default_r256
=====
Loading weights: 100% 146/146 [00:01<00:00, 144.80it/s]
[ 1/112] model.layers.0.self_attn.q_proj
[ 2/112] model.layers.0.self_attn.k_proj
[ 3/112] model.layers.0.self_attn.v_proj
[112/112] model.layers.15.mlp.down_proj
Trainable parameters: 180,355,072 (12.735%)
Training metamath: 24975 examples, 3122 micro-batches, 196 updates, 6 warmup updates.
Train metamath epoch 1/1: 100% 3122/3122 [1:32:37<00:00, 1.81s/it, loss=0.3753, update=196]
```

	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
Training Time (min.)	(1:04:58) ~65min	(1:09:33) ~69min	(1:32:37) ~93min
Trainable parameters	11272192 (11 millions/0.9%)	45008768 (45 millions/3.5%)	180355072 (180millions/ 12.7%)

b. Provide your **observations and explanations** on the training-time trends with different target ranks.

→ higher rank indicates more parameters to train (to fine-tune), causing more runtime as well.

→ The training time looks not linearly proportional to the trainable parameters.

The RANK=64 run has 4x trainable parameters compared to the RANK=16 run, while runtime increase is about +7% (1.35s/it vs 1.26/it).

The RANK=256 run has 4x trainable parameters compared to the RANK=64 run, while runtime increase is about +36% (1.83s/it vs 1.35/it) >> +28% (7%*4).

This shows more the complexity is more than $O(n)$, n =trainable parameters.

Question 3:

a. For each dataset, perform SVD and fine-tune the model by selecting **four ranges of singular values** while fixing **Target Rank to 16**, and report the corresponding **accuracy in bar charts**.

(1.) The **Top** 16 singular values (set "**component**" to "**default**", which already exists in codes)

(2.) 16 consecutive singular values starting at the **25%** position

(3.) 16 consecutive singular values starting at the **50%** position

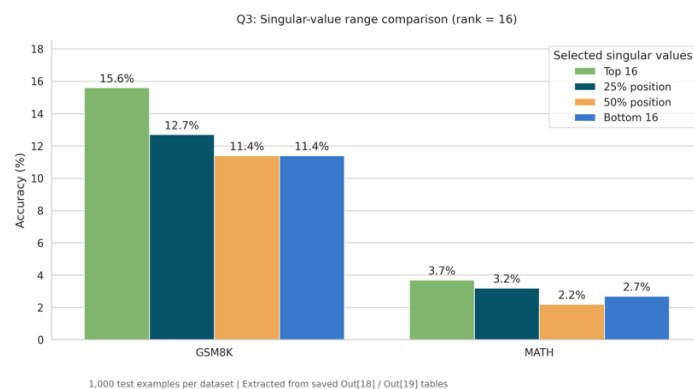
(4.) The **Bottom** 16 singular values

b. Provide your **observations and explanadtions** on the performance trends for both datasets.

→ fine-tuning part: https://github.com/ys6-149286/BigData2026-Assignment1/blob/kaggle_nb/q1-q3-t4.ipynb

→ Plot bar chart only: https://github.com/ys6-149286/BigData2026-Assignment1/blob/kaggle_nb/q3_barchart_plot.ipynb

	method	rank	component	gsm8k_accuracy	math_accuracy	gsm8k_unparsed	math_unparsed
0	fresh	NaN	None	0.035	0.001	15	956
4	pissa	16.0	bottom	0.114	0.027	0	605
1	pissa	16.0	default	0.156	0.037	0	548
3	pissa	16.0	half	0.114	0.022	0	608
2	pissa	16.0	quarter	0.127	0.032	0	590



→ Selecting the top singular values achieved the highest accuracy on both GSM8K (15.6%) and MATH (3.7%). On GSM8K, accuracy decreased to 12.7% at the 25% position and 11.4% for both the 50% position and bottom singular values. On MATH, the corresponding accuracies were 3.2%, 2.2%, and 2.7%. Thus, the trend was not strictly monotonic: the bottom range outperformed the 50% range on MATH by 0.5 percentage points.

→ Selecting different singular values should only affect the initial status of fine-tuning, while most of the parameters of the pretrained model are still freezed, so that it's expected that 4 runs show small amount of accuracy difference.

→ 4 evaluations on MATH dataset got quite bad accuracy compared to GSM8K. There are between 548 and 608 of the 1,000 responses unparsed across the four settings, compared with zero unparsed GSM8K responses. This might be due to the fact that most of the GSM8K outputs are a quite simple number (or a short string), while most of the MATH outputs are a long description with mathematical symbols (maybe latex format). MATH outputs are not that easy to be parsed and all the unparsed ones cannot be considered as accurate answer, causing the low accuracy values on MATH runs. We cannot see significant difference of accuracy in the 4 runs of MATH dataset.

Question 4:

a. Perform SVD on the **query projection ("q_proj") weights** of the LLM layers **0, 7, 15**

→ See https://github.com/ys6-149286/BigData2026-Assignment1/blob/kaggle_nb/finetune_q4-kaggle_nb.ipynb

b. For each layer, please report the **minimum target rank** required to retain **at least 50% cumulative energy**.

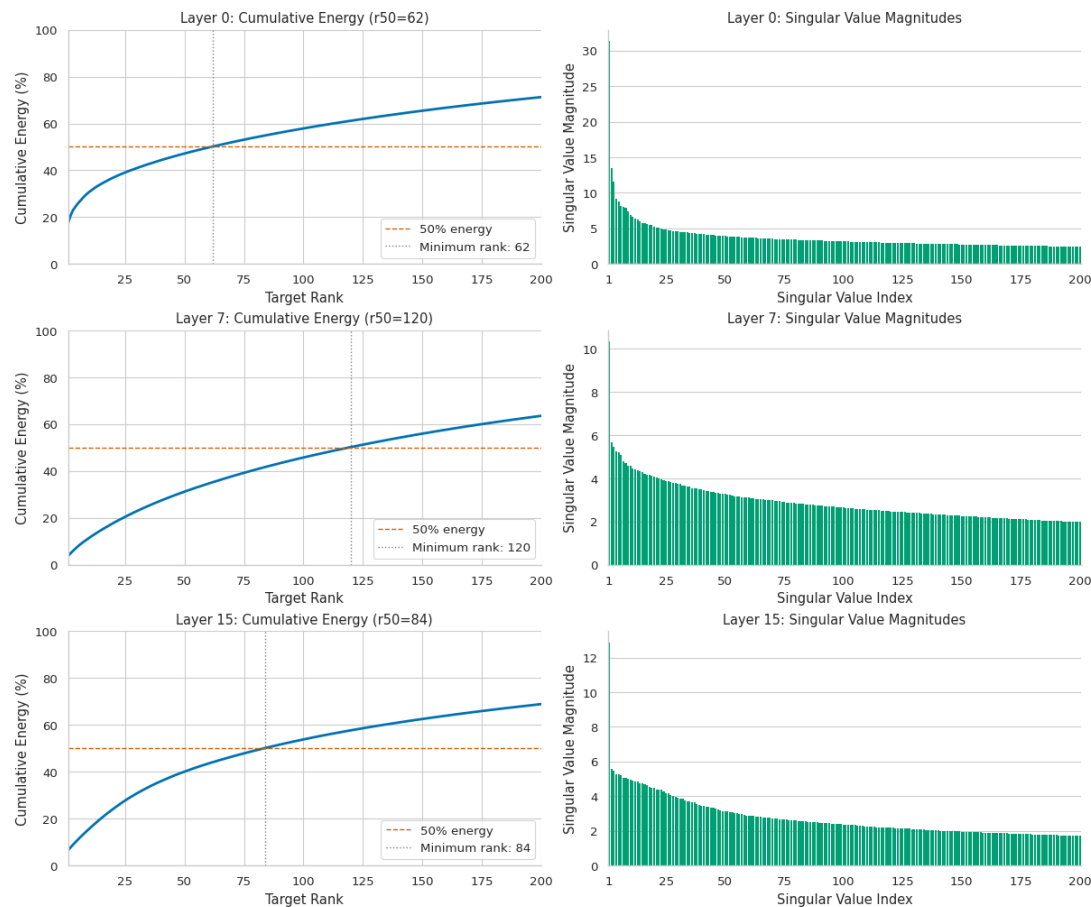
→ layer0: 62, layer7: 120, layer15: 84

layer	weight_shape	minimum_rank_50pct	energy_at_rank_pct	energy_before_rank_pct
0	0 (2048, 2048)	62	50.170053	49.933214
1	7 (2048, 2048)	120	50.201002	49.992613
2	15 (2048, 2048)	84	50.115394	49.870892

c. Please provide the following **visualization for Layer 0, 7, 15, respectively**:(visualize only the **first 200** singular values):

i.Cumulative energy (%) using a **line chart**.

ii.Singular value magnitudes using a **bar chart**.



d. Provide your **observations** for the above results.

→ Sorted by singular value magnitude, the 1st singular value of all 3 layers (layer=0, layer=7, and layer=15) quite dominate most the information. We can see there are obvious peaks when Singular Value Index=1 in the 3 bar charts.

→ Among these 3 layers, the 1st singular value of layer0 has the highest cumulative energy (about 20) compared to other 2 layers. Indicating that the dimensions might be more discriminative than all the other dimensions spanned by other singular vectors (considering all other dimensions of layer0, layer7, and layer15). It's quite straightforward since the input of layer0 is really closed to the input data, i.e. something like word-embedding, which should carry clearer information compared to the input of intermediate hidden layers (such as layer7 or layer 15).

→ The minimum rank to achieve 50% of cumulative energy shows layer0=62, layer7=120, and layer15=84. That indicates that even if we consider more than 60 singular values, we still not achieve the common energy threshold 95% or 99%. Meaning that the intermediate information might be mixed in quite a lot dimensions. So that those hidden parameters might be really hard to be interpreted by human.